Personal Paste / Notepad Project Documentation

Simple Hindi-English Guide

Yeh document project ke model fields, URLs, structure, save flow, slug generation, file upload, aur complete working ko simple language me explain karta hai.

1. Project Overview

Yeh project ek small Django based no-login notepad / pastebin app hai. User jab home page `/` open karta hai, app ek new note create karta hai aur user ko random private URL par redirect kar deta hai.

Jo person note ka URL jaanta hai, woh note read aur edit kar sakta hai. Is project me login system nahi hai, isliye privacy ka main control URL slug hai.

2. Technology Stack

Part	Use
Django	Backend framework, routes, views, model, database handling.
SQLite / MySQL	Default local database SQLite hai. Environment variable `DB_ENGINE=mysql` set karne par MySQL use hota hai.
HTML Template	Editor UI render karta hai.
JavaScript	Auto-save, remote refresh, search, copy URL/text, file upload/delete handle karta hai.
CSS	Dark/light theme aur page design.

3. Folder Structure

Path	Meaning
`manage.py`	Django commands run karne ke liye main file.
`notepad/settings.py`	Project settings: installed apps, database, static/media path.
`notepad/urls.py`	Root URL routing. Admin aur `myapp.urls` include karta hai.

`myapp/models.py`	Database model `Note` yahin define hai.
`myapp/views.py`	Business logic: note create, detail page, API save, file upload/download.
`myapp/urls.py`	App level URLs define karta hai.
`myapp/templates/myapp/note.html`	Note editor page ka HTML.
`myapp/static/myapp/note.js`	Frontend behavior and API calls.
`myapp/static/myapp/note.css`	Styling and theme.
`media/notes/`	Uploaded note files store hote hain.

4. Database Model: Note

Project me main model sirf ek hai: `Note`. Isme note ka content, slug, attachment details, aur timestamps store hote hain.

Field	Type	Details
`slug`	`CharField(max_length=20, unique=True)`	Unique URL identifier. Example: `xqj25ft9`. Har note ka slug different hota hai.
`content`	`TextField(blank=True)`	Note ka text content. Empty bhi ho sakta hai.
`attachment`	`FileField(upload_to=attachment_upload_path, blank=True)`	Note ke saath ek file attach karne ke liye.
`attachment_name`	`CharField(max_length=255, blank=True)`	Original uploaded file ka safe

		name store karta hai.
`attachment_size`	`PositiveBigIntegerField(default=0)`	File size bytes me store hoti hai.
`attachment_uploaded_at`	`DateTimeField(null=True, blank=True)`	File upload time. File expiry calculate karne me use hota hai.
`created_at`	`DateTimeField(auto_now_add=True)`	Note create hone ka time.
`updated_at`	`DateTimeField(auto_now=True)`	Note last update hone ka time.

Ordering: Model ki `Meta` class me `ordering = ["-updated_at"]` hai, matlab latest updated notes first aayenge agar list query chalti hai.

5. Slug Generation Kaise Hota Hai

Slug create karne ka code `views.py` ke `create_note()` function me hai.

1. Allowed characters: lowercase alphabets `a-z` aur digits `0-9`.
2. Slug length: `8` characters.
3. Python `secrets.choice()` use hota hai, jo cryptographically strong random value deta hai.
4. App `Note.objects.create(slug=slug)` se note save karta hai.
5. Agar same slug already exist karta hai, database `IntegrityError` dega because slug unique hai.
6. Collision aane par app dobara try karta hai. Maximum `10` attempts hote hain.
7. 10 attempts fail hone par `RuntimeError` raise hota hai.

Simple words me: home page open karte hi app ek random private link banata hai, note database me save karta hai, aur user ko us link par bhej deta hai.

6. URLs / Routes

URL	Method	View	Work
`/admin/`	GET/POST	Django admin	Admin panel.
`/`	GET	`home`	New note create karta hai aur `/<slug>/` par redirect karta hai.
`/<slug>/`	GET	`note_detail`	Note editor page render karta hai.
`/api/<slug>/`	GET	`note_api`	Latest note data JSON me return karta hai.
`/api/<slug>/`	POST	`note_api`	JSON body se note content save karta hai.
`/api/<slug>/file/`	POST	`note_file`	File upload karta hai. Ek note me ek time par sirf one attachment allowed hai.
`/api/<slug>/file/`	DELETE	`note_file`	Attached file delete karta hai.
`/api/<slug>/file/download/`	GET	`download_note_file`	Attached file download karta hai.

7. Note Save Flow

Frontend Side

1. User textarea me type karta hai.
2. `note.js` input event pakadta hai.
3. Status `Typing...` show hota hai.
4. App `2000 ms` wait karta hai. Is delay ko `SAVE_DELAY_MS` kaha gaya hai.
5. Agar content last saved content se different hai, browser `POST /api/<slug>/` call karta hai.
6. Request JSON body: `{"content": "user text"}`.
7. CSRF token header `X-CSRFToken` ke through send hota hai.

Backend Side

1. `note_api()` slug se note find karta hai.
2. Expired large attachment hai to pehle remove kar sakta hai.
3. JSON body parse hoti hai.
4. Content string hona required hai. Agar string nahi hai to `400` error.

5. Agar new content old content se different hai, only `content` aur `updated_at` save hote hain.
6. Response me note payload return hota hai: slug, content, updated_at, attachment, aur changed flag.

> Save optimized hai: frontend bhi same content save nahi bhejta, aur backend bhi unchanged content par unnecessary database update nahi karta.

8. Remote Refresh / Multi-tab Behavior

Frontend time-to-time `GET /api/<slug>/` call karta hai. Agar note kisi aur tab/browser se update hua ho, current editor automatically latest content load kar leta hai.

Refresh interval user activity ke according change hota hai: active user ke liye fast refresh, idle user ke liye slow refresh. Hidden tab me polling stop ho jati hai.

9. File Upload Flow

1. User `Attach File` button se file select karta hai.
2. JavaScript `FormData` banata hai aur `file` key me selected file append karta hai.
3. Browser `POST /api/<slug>/file/` call karta hai.
4. Backend transaction ke andar note ko `select_for_update()` se lock karta hai, taaki same time par conflicting upload/delete na ho.
5. Agar note me already attachment hai, response `409` aata hai: pehle old file delete karni hogi.
6. File save hoti hai, name, size, upload time database me store hote hain.
7. Response me attachment payload return hota hai.

Attachment Path

Uploaded file ka path `attachment_upload_path()` function banata hai:

`notes/<note-slug>/<random-32-hex><file-extension>`

Example: `notes/xqj25ft9/4af2...c91.pdf`. Isse filename collision avoid hota hai aur files note slug ke folder me organize hoti hain.

10. File Delete And Download

Action	Process
Delete	`DELETE /api/<slug>/file/` call hota hai. Backend actual file delete karta hai, attachment fields empty karta hai, aur JSON me `{"attachment": null}` return karta hai.
Download	`GET /api/<slug>/file/download/` call hota hai. File available hai to `FileResponse` attachment ke form me return hota hai.

Missing file	Agar database me file record hai but disk par file missing hai, backend fields clear karta hai aur `404` return karta hai.

11. Large File Expiry

Is project me file size agar `5 MB` se zyada hai, to file upload ke `3 hours` baad expire ho jati hai.

Expiry check `remove_expired_attachment()` function karta hai. Yeh page open, API get/post, file upload/delete, aur download ke time call hota hai. Expired file milne par `clear_attachment()` actual file aur database fields clear kar deta hai.

12. API Response Structure

Note Payload

Key	Meaning
`slug`	Note ka unique URL slug.
`content`	Note text.
`updated_at`	Last update time ISO format me.
`attachment`	Attachment object ya `null`.
`changed`	POST save ke time optional flag, batata hai content actually changed tha ya nahi.

Attachment Payload

Key	Meaning
`name`	Original file name.
`size`	File size bytes me.
`uploaded_at`	Upload time ISO format me.
`download_url`	File download endpoint.
`expires_at`	Only large files ke liye expiry time, otherwise `null`.

13. UI Features

- Dark/light theme toggle. Theme browser `localStorage` me save hota hai.
- Copy URL button current note link clipboard me copy karta hai.
- Copy Text button note content clipboard me copy karta hai.

- Search box note ke andar matches find karta hai.
- Word count aur character count live update hote hain.
- Save status: `Saved`, `Typing...`, `Saving...`, `Error`.
- Attachment section file name, size, download, and delete actions show karta hai.

14. Database And Media Settings

Setting	Value / Use
`DATABASES`	Default SQLite: `BASE_DIR / "db.sqlite3"`. MySQL optional hai via environment variables.
`STATIC_URL`	`static/`
`STATIC_ROOT`	`BASE_DIR / "staticfiles"`
`MEDIA_ROOT`	`BASE_DIR / "media"`
`TIME_ZONE`	`UTC`
`USE_TZ`	`True`, timestamps timezone aware hote hain.

15. Local Setup Commands

1. `python -m venv notevenv`
2. `source notevenv/bin/activate`
3. `pip install django`
4. `python manage.py migrate`
5. `python manage.py runserver`
6. Browser me open karein: `http://127.0.0.1:8000/`

16. Complete Working In Short

1. User `/` open karta hai.
2. Backend random slug se blank note create karta hai.
3. User `/<slug>/` editor page par redirect hota hai.
4. User type karta hai, frontend 2 seconds baad changed content API ko bhejta hai.
5. Backend content validate karke database me save karta hai.
6. Frontend response ke baad status `Saved` show karta hai.
7. User file attach kar sakta hai; one note me one file allowed hai.
8. Large file 5 MB se zyada ho to 3 hours baad auto cleanup ho sakti hai.
9. Same URL kisi aur ko share karne par woh bhi note read/edit kar sakta hai.

17. Important Points

- Login/authentication nahi hai. URL jiske paas hai, access uske paas hai.
- Slug strong random hai, but URL share carefully karna chahiye.
- CSRF protection enabled hai for browser API requests.
- Only one attachment per note supported hai.
- Production me `DJANGO_SECRET_KEY`, `DJANGO_DEBUG=False`, HTTPS, aur proper host settings use karna chahiye.